BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

Orthogonal Defect Classification on Defects4J Using an LLM-Driven Scientific Approach

Hasin Mahtab Alvee
210042174
Mohammad Nahiyan Kabir
210042168
Md. Sakib Hossain
210042133

Department of Computer Science and Engineering
Islamic University of Technology
April, 2026

Orthogonal Defect Classification on Defects4J Using an LLM-Driven Scientific Approach

Hasin Mahtab Alvee
210042174
Mohammad Nahiyan Kabir
210042168
Md. Sakib Hossain
210042133

Department of Computer Science and Engineering
Islamic University of Technology
April, 2026

Declaration of Candidate

This is to certify that the work presented in this thesis is the outcome of the analysis and experiments carried out by **Hasin Mahtab Alvee**, **Mohammad Nahiyan Kabir**, and **Md. Sakib Hossain** under the supervision of **Lutfun Nahar Lota**, Assistant Professor, Department of Computer Science and Engineering and co-supervision of **Ishmam Tashdeed**, Lecturer, Department of Computer Science and Engineering, Islamic University of Technology, Dhaka, Bangladesh. It is also declared that neither this thesis nor any part of it has been submitted anywhere else for any degree or diploma. Information derived from the published and unpublished work of others have been acknowledged in the text and a list of references is given.

Lutfun Nahar Lota
Assistant Professor
Department of Computer Science and Engineering
Islamic University of Technology (IUT)
Date: April 22, 2026

Ishmam Tashdeed
Lecturer
Department of Computer Science and Engineering
Islamic University of Technology (IUT)
Date: April 22, 2026

Hasin Mahtab Alvee
Student ID: 210042174
Date: April 22, 2026

Mohammad Nahiyan Kabir
Student ID: 210042168
Date: April 22, 2026

Md. Sakib Hossain
Student ID: 210042133
Date: April 22, 2026

Dedicated to our families, friends, and mentors who supported us throughout this journey

Contents

1 Introduction **1**

1.1 Defects4J: The Benchmark Dataset 1

1.2 Motivation 2

1.3 Research Objectives 2

1.4 Contributions 3

1.5 Report Organization 3

2 Related Works **4**

2.1 Existing Defects4J Classification Attempts 4

2.2 LLM-Based Defect Classification 5

2.3 The Taxonomy Problem 5

2.4 Orthogonal Defect Classification (ODC) 6

2.5 Summary 6

3 Problem Statement **8**

3.1 Limitations of Existing Approaches 8

3.2 The Research Gap 8

3.3 Problem Statement 9

4 Proposed Methodology **10**

4.1 Pipeline Architecture Overview 10

4.2 ODC and Defects4J Artifact Mapping 11

4.3 Evidence Collection 12

4.4 Evidence Modes: Pre-fix vs Post-fix 13

4.5 Prompt Design 14

4.6 Scientific Debugging Protocol 15

4.7 Classification Pipeline and Validation 16

4.8 Evaluation Framework 17

4.9 Summary 18

5 Experimental Design and Data Collection **19**
5.1 Dataset: Defects4J 19
5.2 Experimental Setup 20
5.2.1 Evidence Collection Configuration 20
5.2.2 LLM Configuration 21
5.2.3 Evaluation Configuration 21
5.3 Data Collection Plan 21
5.4 Multi-Fault Context 22

6 Results and Analysis **23**
6.1 Classification Results Overview 23
6.2 Analysis of Type-Shifted Bugs 23
6.3 Why Types Diverge 24
6.4 Project Coverage 25
6.5 Summary 25

7 Conclusion **26**
7.1 Summary 26
7.2 Limitations 27
7.3 Future Work 27
7.4 Closing Remarks 28

References **29**

List of Figures

4.1 Pipeline architecture overview showing the three stages: Collect, Classify, and Compare . . . 11
4.2 Natural mapping between Defects4J artifacts and ODC opener/closer attributes . . . 11
4.3 Evidence collection process: eight steps from metadata query to context serialization . . . 12
4.4 Pre-fix vs Post-fix evidence modes with their corresponding ODC perspectives . . . 13
4.5 Six components of the system prompt: ODC taxonomy, anti-bias rules, scientific debugging, diagnostic tree, JSON schema, and few-shot examples . . . 14
4.6 Scientific debugging protocol with five steps and the 7-question diagnostic decision tree . . . 15
4.7 Classification pipeline flow from context loading through validation to output . . . 16
4.8 4-tier evaluation framework: Strict Match, Top-2 Match, Family Match, and Cohen's Kappa . . . 17

List of Tables

5.1 Defects4J projects and their active bug counts 20

6.1 Preliminary evaluation results across the 4-tier framework (34 bug pairs) 23

List of Abbreviations

APR	Automated Program Repair
API	Application Programming Interface
CFG	Control Flow Graph
CLI	Command Line Interface
CoT	Chain-of-Thought
D4J	Defects4J
DFG	Data Flow Graph
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
ODC	Orthogonal Defect Classification
SDLC	Software Development Life Cycle
SVM	Support Vector Machine

Acknowledgement

We would like to express our sincere gratitude to our supervisor, Lutfun Nahar Lota, and our co-supervisor, Ishmam Tashdeed, for their continuous guidance and support throughout this research. Their expertise and encouragement helped shape this work from concept to completion.

We are also grateful to the Network and Data Analysis Group (NDAG) Lab at IUT for providing the research environment and resources that made this thesis possible.

Abstract

Bug classification plays a key role in software quality assurance, helping developers understand defect patterns and improve their development processes. However, no study has ever fully labeled all bugs in the Defects4J benchmark dataset using a principled, root-cause taxonomy. Existing classification efforts cover only small subsets (50 to 488 bugs), rely on structural taxonomies that describe how bugs look rather than why they occur, or depend on post-fix code that is not available during real-world triage. This thesis presents an automated pipeline that classifies Defects4J bugs into seven Orthogonal Defect Classification (ODC) defect types using Large Language Model (LLM) reasoning guided by a scientific debugging protocol. The pipeline collects multi-modal evidence from Defects4J artifacts, including bug reports, stack traces, failing tests, and source code snippets, and constructs structured prompts that combine the ODC taxonomy with Zeller's scientific debugging methodology. Classification operates in two evidence modes: pre-fix (symptoms only) and post-fix (with the actual code fix), enabling a built-in evaluation through comparison. Preliminary analysis of 34 bug pairs across 17 projects shows 73.5% strict type match, 94.1% top-2 match, and 97.1% family match between the two modes, demonstrating robust classification consistency that aligns with the expected 20 to 30% human disagreement ceiling reported in ODC literature.

Chapter 1

Introduction

Software bugs are an unavoidable part of the development process. Even the most carefully written programs contain defects that cause unexpected behavior, crashes, or incorrect results. Identifying these bugs is only the first step. Understanding what kind of bug it is, why it happened, and how it should be fixed is equally important for improving software quality over time [3].

In practice, developers spend a significant portion of their time triaging and debugging reported issues. Bug categorization helps make this process more efficient by grouping defects into meaningful categories that guide debugging strategies and resource allocation [1]. Early prediction of a bug's category from its report helps teams prioritize work before even looking at the code [8]. Structured categorization also provides actionable measurements that managers and developers can use to improve their processes [4].

1.1 Defects4J: The Benchmark Dataset

Defects4J is a curated database of real, reproducible Java bugs paired with developer-written patches and regression test suites [11]. It has grown to include over 864 active bugs across 17 Java projects and serves as the standard benchmark for empirical software engineering research, including fault localization, automated program repair, and testing studies [6], [16].

Each bug in Defects4J comes with a rich set of artifacts: the original bug report, triggering tests that expose the bug, the buggy source code, the fixed source code, and code diffs showing exactly what changed. These artifacts make Defects4J particularly well suited for automated analysis because they provide multiple perspectives on each

defect.

1.2 Motivation

Despite the widespread use of Defects4J in software engineering research, no study has ever fully classified all of its bugs using a principled, root-cause taxonomy. Existing classification efforts fall into three categories, each with significant limitations.

First, structural approaches like the work of Van der Spuy and Fischer [19] classify bugs based on control-flow and data-flow graph representations. While this covers 488 bugs across 7 projects, it captures how bugs manifest syntactically rather than why they exist semantically.

Second, patch-level taxonomies like the Defects4J Dissection by Sobreira et al. [17] classify repair actions for 395 bugs from an older version of the dataset. This describes fix patterns but does not identify root causes.

Third, small-sample inspections like the work of Xuan, Monperrus, et al. [20] manually analyze 50 randomly selected bugs. While detailed, this approach does not scale.

None of these approaches apply a semantic, root-cause taxonomy to the full dataset. This gap motivates the need for an automated, scalable classification approach that goes beyond surface-level structural patterns.

1.3 Research Objectives

This thesis addresses the following four research objectives:

RO-1: Build an automated pipeline that classifies Defects4J bugs into ODC defect types using LLM reasoning guided by structured scientific debugging.

RO-2: Collect bug evidence in two modes, pre-fix (symptoms only) and post-fix (with the actual code fix), mapping Defects4J artifacts to ODC opener and closer attributes.

RO-3: Apply Zeller's scientific debugging methodology as an LLM prompting strategy to produce explainable, step-by-step classifications.

RO-4: Design a 4-tier evaluation framework that compares pre-fix and post-fix classifications using strict match, top-2 match, family match, and Cohen's Kappa.

1.4 Contributions

The main contributions of this thesis are:

- A fully automated, end-to-end pipeline that collects multi-modal evidence from Defects4J and produces machine-readable ODC classifications with full reasoning traces.
- A dual-mode evidence collection system (pre-fix and post-fix) that mirrors ODC's opener/closer model and enables built-in evaluation.
- A structured prompting approach that encodes the ODC taxonomy and Zeller's scientific debugging protocol into LLM prompts, producing explainable classifications.
- A 4-tier evaluation framework that respects the inherent ambiguity in defect classification rather than demanding perfect agreement.

1.5 Report Organization

The rest of this report is organized as follows. Chapter 2 reviews the related literature on Defects4J classification, LLM-based bug categorization, and the ODC framework. Chapter 3 defines the research gap and problem statement. Chapter 4 describes the proposed methodology in detail. Chapter 5 explains the experimental design and data collection approach. Chapter 6 presents the preliminary results and analysis. Chapter 7 provides the project timeline. Finally, Chapter 8 concludes the report with a summary, limitations, and future work.

Chapter 2

Related Works

This chapter reviews the existing literature that forms the foundation of our research. We organize the review around four themes: existing attempts at classifying Defects4J bugs, the emergence of LLM-based defect classification, the challenge of choosing the right taxonomy, and the Orthogonal Defect Classification framework that our pipeline adopts.

2.1 Existing Defects4J Classification Attempts

Several studies have attempted to classify bugs in the Defects4J dataset, each using a different approach and covering a different subset of the data.

The most recent effort is by Van der Spuy and Fischer [19], who manually classified 488 Defects4J bugs into six control-flow and two data-flow categories based on their graph representations. This structural approach reveals how bugs change the program's execution paths, but it does not explain the semantic root cause of the defect. For example, a missing null check and a completely absent feature might both appear as similar control-flow changes, even though they represent fundamentally different types of problems.

Sobreira et al. [17] performed an earlier classification of 395 bugs from Defects4J version 1.2, focusing on repair actions and repair patterns. Their work found that 43% of patches involve conditional block changes. While this provides useful information about fix patterns, it describes what developers changed rather than why the bug existed in the first place.

Xuan, Monperrus, et al. [20] conducted a manual inspection of 50 randomly selected

Defects4J bugs, identifying fixability patterns and assessing the quality of test-suite artifacts. While this small-sample study reveals important details, the authors themselves acknowledge the limited external validity of examining only 50 bugs.

These three studies collectively represent structural classification, patch-level classification, and small-sample inspection. None of them applies a semantic, root-cause taxonomy like ODC to the full Defects4J dataset.

2.2 LLM-Based Defect Classification

Recent research has shown that Large Language Models can effectively classify software defects when given appropriate prompting strategies.

Colavito et al. [5] demonstrated that LLMs can classify issue reports (closely related to bug reports) with reasonable accuracy, validating the basic hypothesis that LLMs are suitable for defect classification tasks. Koyuncu [13] extended this by using prompt engineering techniques, including few-shot prompting, to achieve fine-grained bug report categorization. Their work is the most directly relevant prior study to our approach.

Chain-of-thought prompting has also proven effective for structured classification tasks. Nong et al. [14] showed that CoT prompting improves LLM classification accuracy by over 500% on vulnerability analysis tasks, suggesting that structured reasoning significantly helps LLMs perform better on classification problems.

Gao et al. [7] demonstrated that LLMs can generate structured, ODC-aligned root cause explanations from bug reports and stack traces, showing that LLMs can reason about semantic defect categories when provided with appropriate guidance.

Compared to traditional machine learning approaches, LLMs offer a key advantage: they are code-aware. Earlier automated ODC systems using SVM and Naive Bayes classifiers achieved only 77 to 82% accuracy and struggled with sparse text inputs [9], [18]. LLMs overcome these limitations because they can reason about code structure, test semantics, and error messages simultaneously, without requiring labeled training data.

2.3 The Taxonomy Problem

While LLMs show promise for defect classification, using them without a structured taxonomy leads to unreliable results.

Gao et al. [7] noted that LLMs hallucinate heavily when classifying bugs without concrete stack traces or a guiding taxonomy. Without standardization, different LLM runs produce different category labels for the same bug [13]. Machine learning-based bug classification studies also suffer from a proliferation of incompatible taxonomies, making it impossible to compare results across studies [1]. Classification without a principled framework produces vague labels with no operational value [18].

This problem motivates the need for a well-established, mathematically rigorous taxonomy to constrain and guide LLM outputs. That taxonomy is Orthogonal Defect Classification.

2.4 Orthogonal Defect Classification (ODC)

ODC was introduced by Chillarege et al. [4] as a framework for classifying software defects using orthogonal (independent) attributes that provide actionable, in-process measurements. The framework defines eight attributes organized into two phases:

- **Opener attributes** (recorded when the defect is found): Activity, Trigger, and Impact. These capture the discovery context: what was being done, what condition exposed the defect, and what effect it had.
- **Closer attributes** (recorded when the defect is fixed): Defect Type, Target, Qualifier, Age, and Source. These capture the repair context: what kind of mistake it was, where it was located, and how it manifested.

The central attribute for our work is the Defect Type, which classifies the root cause mechanism into seven categories for Design/Code targets: Algorithm/Method, Assignment/Initialization, Checking, Timing/Serialization, Function/Class/Object, Interface/OO Messages, and Relationship. These seven types are grouped into two families: Control and Data Flow (the first four) and Structural (the last three).

ODC remains actively used in modern research. Pan et al. [15] applied ODC to classify defects in LLM-generated code in 2024, demonstrating that the taxonomy generalizes beyond human-written software. The official ODC v5.2 specification from IBM [10] continues to serve as the reference standard for the framework.

2.5 Summary

The literature reveals a clear gap: no study has applied a semantic, root-cause taxonomy like ODC to the full Defects4J dataset using automated classification. Existing

work either covers small subsets, uses structural-only taxonomies, or requires labeled training data that Defects4J lacks. LLMs offer a viable path forward, but they need a structured taxonomy to produce consistent, actionable results. Our thesis addresses this gap by combining LLM reasoning with ODC constraints and a scientific debugging protocol.

Chapter 3

Problem Statement

This chapter defines the specific research gap that our thesis addresses and formally states the problem we aim to solve.

3.1 Limitations of Existing Approaches

Current attempts at classifying Defects4J bugs suffer from five critical limitations:

1. **Coverage Gap:** No study has fully labeled all 864+ Defects4J bugs. Existing work covers only 50 to 488 bugs [17], [19], [20].
2. **Structural-Only Taxonomy:** Control-flow and data-flow graph approaches capture how bugs manifest syntactically, not why they exist semantically [19].
3. **Patch-Level Focus:** Repair-action classification describes fix patterns, not the underlying root cause mechanism [17].
4. **Post-Fix Dependency:** Existing ML approaches require labeled training data (which Defects4J does not have for ODC) or access to post-fix code [18].
5. **Descriptive, Not Diagnostic:** Current labels are too low-level or inconsistent for decision-making in triage and quality assurance [3].

3.2 The Research Gap

No study has produced a context-aware, semantically grounded, scalable classification of the full Defects4J dataset. Specifically, the gap requires a solution that is:

- **Context-aware:** Using multi-modal evidence from code, tests, bug reports, and stack traces rather than relying on a single information source.
- **Semantic:** Classifying defects by their root cause mechanism, not by syntactic patterns.
- **Taxonomy-constrained:** Using a principled framework like ODC to prevent hallucination and ensure comparability across studies.
- **Scalable:** Using LLM-based reasoning to classify the full dataset without requiring manual labeling.

3.3 Problem Statement

Given these limitations, we define the problem as follows: *How can we build an automated system that classifies Defects4J bugs into ODC defect types using only pre-fix evidence (symptoms), while also providing a mechanism to evaluate the quality of those classifications by comparing them against post-fix classifications?*

This problem naturally decomposes into the four research objectives stated in Chapter 1 (RO-1 through RO-4).

Chapter 4

Proposed Methodology

This chapter describes the design and implementation of our classification pipeline. The pipeline has three main stages: evidence collection, LLM-based classification, and evaluation through comparison. We describe each stage in detail, along with the ODC artifact mapping, prompt design, and scientific debugging protocol that make the system work.

4.1 Pipeline Architecture Overview

The pipeline follows a file-oriented, synchronous architecture with no database, queue, or asynchronous orchestration. It is organized into three sequential stages, as shown in Figure 4.1.

1. **COLLECT:** The Defects4J CLI is used to gather multi-modal evidence from a bug, producing a structured `context.json` file.
2. **CLASSIFY:** The context is used to build structured prompts that are sent to an LLM, producing a `classification.json` file.
3. **COMPARE:** Pre-fix and post-fix classifications are compared using a multi-tier evaluation framework.

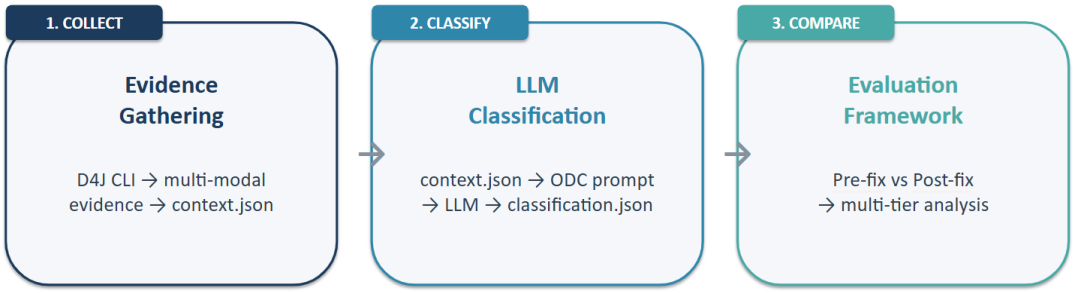

Figure 4.1: Pipeline architecture overview showing the three stages: Collect, Classify, and Compare

Key architectural principles include multi-provider LLM support (Gemini, OpenRouter, OpenAI-compatible APIs), pre-fix evidence as the default mode, and hidden oracle exclusion where the list of modified classes never enters the LLM prompt to prevent data leakage. The design is inspired by AutoSD [12], which encoded Zeller's scientific debugging into an LLM pipeline for fault localization [21].

4.2 ODC and Defects4J Artifact Mapping

The pipeline maps Defects4J artifacts to ODC attributes following the framework's two-phase opener/closer model, as shown in Figure 4.2.

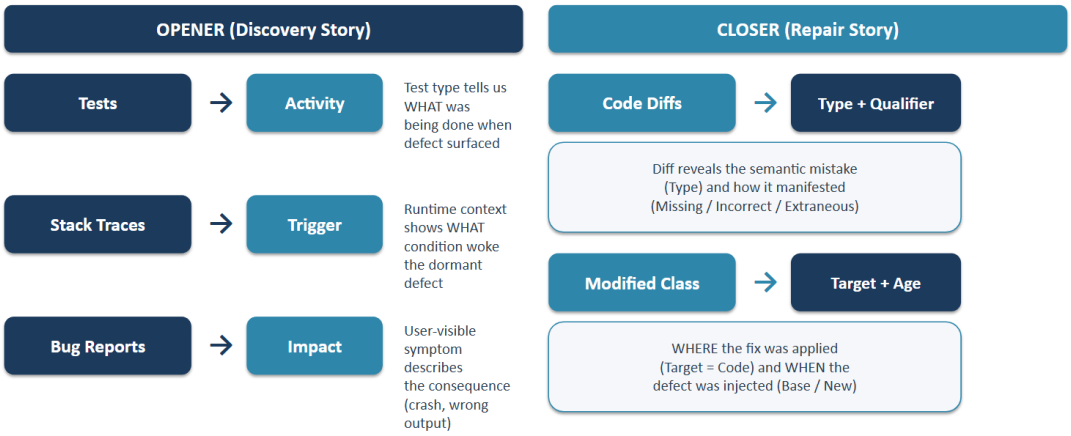

Figure 4.2: Natural mapping between Defects4J artifacts and ODC opener/closer attributes

On the opener side (discovery story):

- **Tests** map to **Activity**, because the test type (unit test, function test, system test) directly tells us what was being performed when the defect was found.
- **Stack Traces** map to **Trigger**, because the runtime context captured in the trace shows the specific condition that woke up the dormant defect.

- **Bug Reports** map to **Impact**, because they describe the user-visible symptom and consequence, such as a crash or wrong output.

On the closer side (repair story):

- **Code Diffs** map to **Defect Type + Qualifier**, because the diff reveals the semantic nature of the mistake and how it manifested (Missing, Incorrect, or Extraneous).
- **Modified Classes** map to **Target + Age**, because they show where the fix was applied and when the defect was likely injected.

This clean separation ensures that subjective discovery data is never mixed with objective technical repair data, following ODC's fundamental design principle [4].

4.3 Evidence Collection

The evidence collection stage gathers multi-modal bug information from Defects4J, as shown in Figure 4.3. The process follows eight sequential steps:

Figure 4.3: Evidence collection process: eight steps from metadata query to context serialization

1. **Query D4J Metadata:** Retrieve project information, bug details, and the report URL.
2. **Fetch Bug Report:** Download the bug report content from JIRA, GitHub, or a generic web page for opener-side evidence.
3. **Checkout and Compile:** Check out the buggy version and compile it.
4. **Run Tests and Parse Failures:** Execute the test suite and extract test names, stack traces, and exception messages from the results.

5. **Select Suspicious Frames:** Filter out framework, JDK, and build-tool frames, prioritizing project source code frames.
6. **Extract Code Snippets:** Pull production code (plus or minus 12 lines around each suspicious frame) and test source code (plus or minus 18 lines) to show expected versus actual behavior.
7. **Coverage (optional):** Run Cobertura-style coverage analysis and parse the resulting XML for line and branch execution rates.
8. **Serialize context.json:** Package all collected evidence into a structured JSON file for the classification stage.

An important design decision is hidden oracle exclusion: the list of modified classes (from `classes.modified`) is stored in a `hidden_oracles` field and deliberately excluded from the LLM prompt to prevent data leakage [16].

4.4 Evidence Modes: Pre-fix vs Post-fix

The pipeline operates in two evidence modes to support both realistic classification and evaluation, as shown in Figure 4.4.

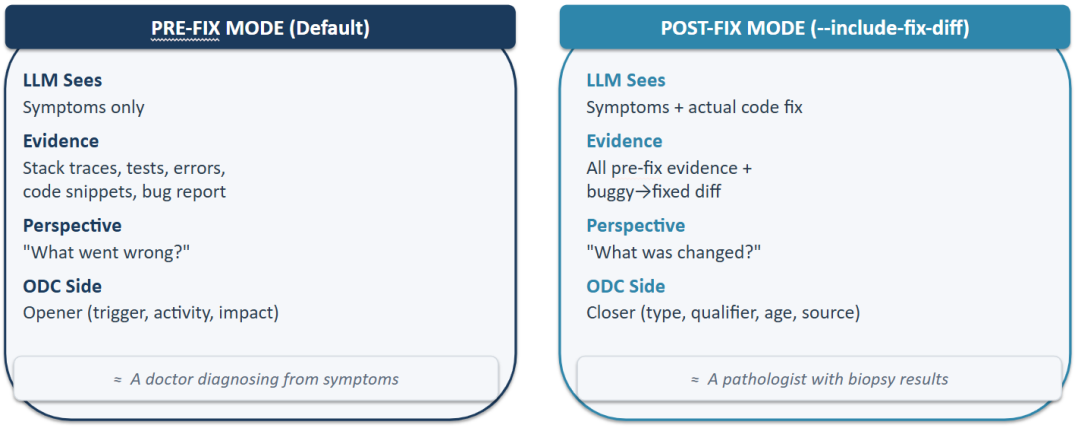

Figure 4.4: Pre-fix vs Post-fix evidence modes with their corresponding ODC perspectives

Pre-fix mode (the default) gives the LLM only symptoms: stack traces, test failures, error messages, code snippets, and bug report content. The LLM sees what went wrong and must reason about the root cause without knowing the fix. This mirrors opener-side ODC evidence (trigger, activity, impact). Conceptually, this is like a doctor diagnosing a patient from symptoms alone.

Post-fix mode (enabled with the `-include-fix-diff` flag) adds the actual buggy-to-fixed code diff as oracle information. The LLM can now see exactly what was changed, giving it closer-side ODC evidence (defect type, qualifier, age, source). This is like a pathologist with biopsy results who can see the exact problem.

ODC's opener/closer model naturally requires evidence from both perspectives [4]. Pre-fix captures opener evidence, and post-fix captures closer evidence. This dual-mode design enables a built-in evaluation strategy: if the pipeline classifies correctly from symptoms alone, the pre-fix result should broadly agree with the post-fix result.

4.5 Prompt Design

The classification prompt consists of six components that work together to guide the LLM toward consistent, explainable ODC classifications. The prompt structure is shown in Figure 4.5.

Figure 4.5: Six components of the system prompt: ODC taxonomy, anti-bias rules, scientific debugging, diagnostic tree, JSON schema, and few-shot examples

1. **ODC Taxonomy:** The seven defect types with definitions, diagnostic indicators, and contrastive "distinguish from" guidance that helps the LLM differentiate between similar types.

2. **Anti-Bias Rules:** Explicit instructions to avoid defaulting to Function/Class/Object, which is the most general structural type. The prompt forces the LLM to consider simpler explanations first.

3. **Scientific Debugging Protocol:** A structured five-step process (Hypothesize, Observe, Predict, Examine, Conclude) based on Zeller's scientific debugging methodology [21], adapted for LLM prompting following the approach of Kang et al. [12].

4. **Diagnostic Decision Tree:** Seven questions, each mapping to one ODC type. Function/Class/Object is deliberately placed last to force consideration of simpler types first [14].

5. **JSON Schema:** A structured output format that requires reasoning traces, confidence scores, alternative type considerations, and ODC attribute fields.

6. **Few-Shot Examples (5):** Contrastive examples that demonstrate the pattern: Symptom, Code, Classification, and why it is NOT another type [13].

4.6 Scientific Debugging Protocol

The scientific debugging protocol is the core reasoning strategy encoded into the LLM prompt. It adapts Zeller's five-step scientific debugging approach [21] for ODC classification, as shown in Figure 4.6.

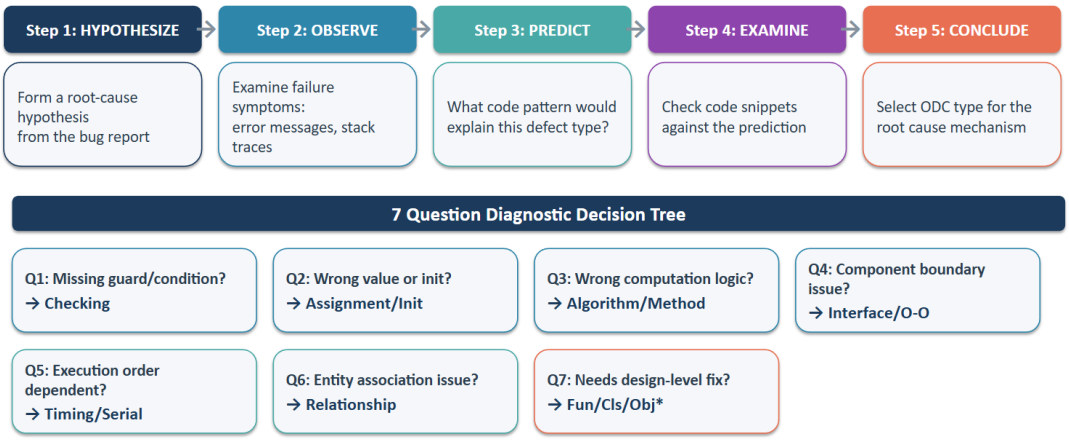

Figure 4.6: Scientific debugging protocol with five steps and the 7-question diagnostic decision tree

The five steps are:

1. **Hypothesize:** Form a root-cause hypothesis from the bug report.

2. **Observe:** Examine failure symptoms including error messages and stack traces.

3. **Predict:** Determine what code pattern would explain this defect type.

4. **Examine:** Check code snippets against the prediction.

5. **Conclude:** Select the ODC type that best explains the root cause mechanism.

The diagnostic decision tree consists of seven questions (Q1 through Q7), each corresponding to one ODC type. Q7 ("Does this need a design-level fix?") is deliberately

placed last because Function/Class/Object should only be selected when all simpler types have been ruled out. This ordering was inspired by the anti-bias principles in chain-of-thought prompting research [14].

4.7 Classification Pipeline and Validation

The classification stage takes a `context.json` file and produces a `classification.json` file through a structured process, as shown in Figure 4.7.

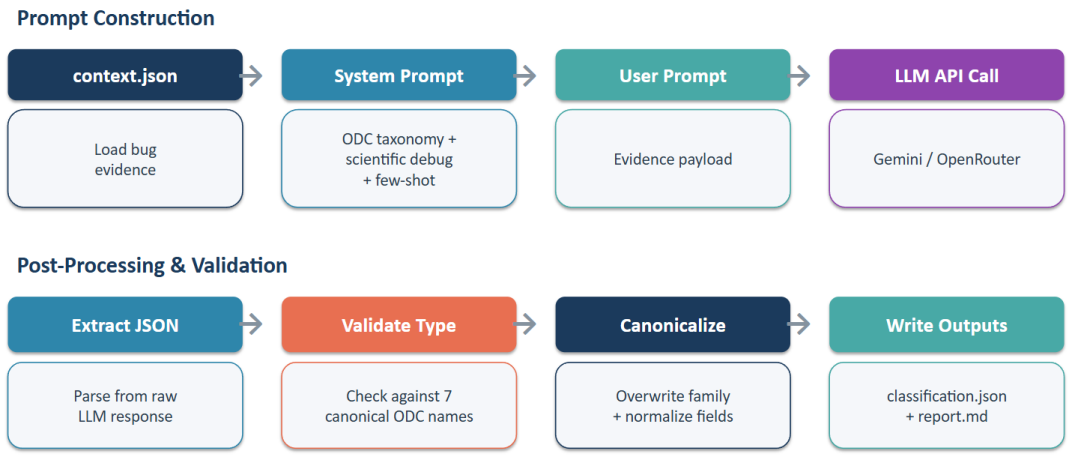

Figure 4.7: Classification pipeline flow from context loading through validation to output

The process consists of:

1. **Prompt Construction:** Load the bug evidence, build the system prompt (ODC taxonomy + scientific debugging + few-shot examples), and construct the user prompt (evidence payload with hidden oracles filtered out).
2. **LLM API Call:** Send the prompt to the configured LLM provider (Gemini, OpenRouter, or OpenAI-compatible).
3. **Response Parsing:** Extract the JSON classification from the raw LLM response, handling fenced code blocks and extra text.
4. **ODC Validation:** Validate the `odc_type` against the canonical seven types. The family is always recomputed from the type (not trusted from the model) to ensure consistency.
5. **Output Generation:** Write the validated `classification.json` and optionally generate a human-readable `report.md`.

The pipeline supports multi-provider LLM backends. Gemini uses a response JSON schema for structured output, while OpenRouter and OpenAI-compatible providers use chat completions with post-hoc JSON parsing. Transient HTTP errors (429, 500, 502, 503) are retried with exponential backoff.

4.8 Evaluation Framework

The pipeline uses a 4-tier evaluation framework to compare pre-fix and post-fix classifications, as shown in Figure 4.8. This multi-tier approach is necessary because even human ODC experts disagree on 20 to 30% of bugs [4].

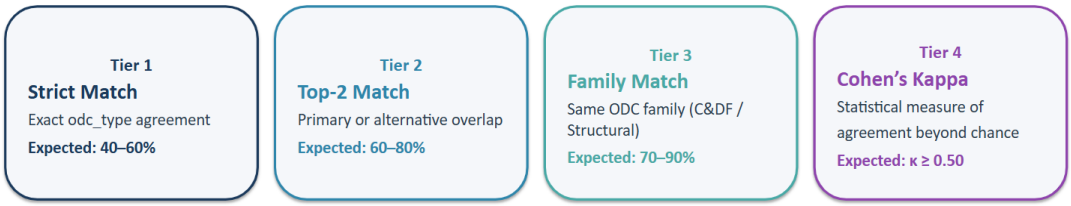

Figure 4.8: 4-tier evaluation framework: Strict Match, Top-2 Match, Family Match, and Cohen's Kappa

The four tiers are:

1. **Strict Match:** Do the pre-fix and post-fix classifications assign the exact same ODC type? This is the most demanding metric.
2. **Top-2 Match:** Is the pre-fix primary type found in the post-fix alternative types (or vice versa)? This captures cases where the LLM correctly identified the type but ranked it second.
3. **Family Match:** Do both classifications fall within the same family (Control and Data Flow or Structural)? This captures broad semantic agreement even when the specific type differs.
4. **Cohen's Kappa:** A statistical measure of inter-rater reliability that accounts for chance agreement, computed only in batch comparisons [18].

This tiered approach captures both exact agreement and meaningful near-agreement, matching industry evaluation practice [4], [18].

4.9 Summary

The proposed methodology combines three ideas: multi-modal evidence from Defects4J, ODC's principled taxonomy, and Zeller's scientific debugging protocol, all encoded into structured LLM prompts. The dual-mode (pre-fix/post-fix) design provides a built-in evaluation mechanism, and the 4-tier comparison framework respects the inherent ambiguity in defect classification. The next chapters describe the experimental setup and preliminary results from applying this methodology.

Chapter 5

Experimental Design and Data Collection

This chapter describes how we set up our experiments, what data we collected, and how we configured the pipeline for the classification runs.

5.1 Dataset: Defects4J

The experiment uses the Defects4J benchmark dataset version 2.0, which contains over 864 active bugs across 17 open-source Java projects [11]. Table 5.1 shows the projects included in the dataset.

Table 5.1: Defects4J projects and their active bug counts

Project	Description	Active Bugs
Chart	JFreeChart plotting library	26
Cli	Apache Commons CLI	40
Closure	Google Closure Compiler	174
Codec	Apache Commons Codec	18
Collections	Apache Commons Collections	4
Compress	Apache Commons Compress	47
Csv	Apache Commons CSV	16
Gson	Google Gson	18
JacksonCore	Jackson JSON core	26
JacksonDatabind	Jackson databinding	112
JacksonXml	Jackson XML	6
Jsoup	HTML parser	93
JxPath	Apache JXPath	22
Lang	Apache Commons Lang	64
Math	Apache Commons Math	106
Mockito	Mocking framework	38
Time	Joda-Time	26
Total		**835+**

Each bug provides the following artifacts used by our pipeline: bug reports (from JIRA, GitHub, or project trackers), triggering test cases, buggy source code, fixed source code, and buggy-to-fixed diffs. Defects4J also provides CLI tools for checkout, compilation, test execution, and coverage analysis.

5.2 Experimental Setup

5.2.1 Evidence Collection Configuration

For each bug, the pipeline collects evidence in two modes:

- **Pre-fix:** Symptoms only (bug reports, stack traces, failing tests, code snippets). No access to the fix.
- **Post-fix:** All pre-fix evidence plus the actual buggy-to-fixed code diff (enabled with `-include-fix-diff`).

Stack frame filtering is configured to remove framework frames (JUnit, TestNG, Hamcrest), JDK internal frames, and build-tool frames. Only project source frames are retained. Production code snippets capture plus or minus 12 lines around each suspicious frame. Test source snippets capture plus or minus 18 lines and are capped at the first 3 failing tests.

5.2.2 LLM Configuration

The classification uses the scientific prompt style with the following LLM configuration:

- **Provider:** Google Gemini (primary), with OpenRouter as secondary
- **Prompt style:** Scientific (includes the 5-step debugging protocol and 7-question decision tree)
- **Few-shot examples:** 5 contrastive examples included in every prompt
- **Output format:** Structured JSON with reasoning traces, confidence scores, and alternative type considerations

5.2.3 Evaluation Configuration

Each bug produces two classification files: one from pre-fix evidence and one from post-fix evidence. The `compare-batch` command automatically discovers matching pairs and computes all four evaluation tiers (strict match, top-2 match, family match, and Cohen's Kappa).

5.3 Data Collection Plan

The full experiment targets all 864+ active Defects4J bugs. For the preliminary analysis presented in this report, we have completed 34 bug pairs across 17 projects. The full dataset run is planned as a batch operation covering all remaining bugs.

The data collection process for each bug follows these steps:

1. Run `d4j-odc collect` to generate pre-fix `context.json`
2. Run `d4j-odc classify` on the pre-fix context
3. Run `d4j-odc collect -include-fix-diff` to generate post-fix `context.json`
4. Run `d4j-odc classify` on the post-fix context

5. Run `d4j-odc compare` to evaluate the pair

After all individual pairs are processed, `compare-batch` aggregates the results and computes dataset-wide statistics.

5.4 Multi-Fault Context

An important consideration for evaluation is that Defects4J bugs do not exist in isolation. Research by Callaghan [2] shows that real programs average approximately 9.2 co-existing faults per version. Our pipeline includes a multi-fault awareness module that can query the defects4j-mf dataset to identify how many other faults co-exist with each bug. This context helps explain classification divergence in cases where symptom evidence is contaminated by multiple overlapping faults.

Chapter 6

Results and Analysis

This chapter presents the preliminary results from our analysis of 34 bug pairs across 17 Defects4J projects and discusses their implications.

6.1 Classification Results Overview

The preliminary analysis of 34 pre-fix vs post-fix classification pairs produces the following results across the 4-tier evaluation framework, as shown in Table 6.1.

Table 6.1: Preliminary evaluation results across the 4-tier framework (34 bug pairs)

Evaluation Tier	Match Count	Rate
Strict Match (exact same type)	25/34	73.5%
Top-2 Match (including alternatives)	32/34	94.1%
Family Match (same family)	33/34	97.1%

These results show that while the exact type does not always match between pre-fix and post-fix classifications, the disagreement is almost always within the same family of defect types. The 73.5% strict match rate is consistent with ODC literature, which reports that even human experts disagree on 20 to 30% of defect classifications [4].

6.2 Analysis of Type-Shifted Bugs

Out of 34 bug pairs, 9 showed a type shift (different ODC type between pre-fix and post-fix). Of these 9:

- 7 bugs showed cross-alternative overlap, meaning the pre-fix primary type appeared in the post-fix alternative types or vice versa. This indicates that the LLM identified the correct type but ranked it differently depending on available evidence.
- 2 bugs (JacksonCore-24 and Math-34) showed no alternative overlap, but both still matched at the family level.

This pattern is expected and scientifically explainable. Pre-fix classification works from symptoms (like a doctor diagnosing from symptoms), while post-fix classification works from the actual fix (like a pathologist with biopsy results). The two perspectives naturally emphasize different aspects of the same underlying defect.

6.3 Why Types Diverge

The 26.5% type-shift rate reflects the expected difference between symptom-based and cause-based classification, not a failure of the pipeline. Three main factors explain why pre-fix and post-fix classifications can disagree:

1. **Evidence Asymmetry:** Pre-fix evidence shows symptoms (what went wrong), while post-fix evidence shows the cause (what was changed). Symptoms and causes can suggest different ODC types because a single root cause can produce multiple different symptoms.
2. **ODC Boundary Ambiguity:** Some bugs legitimately sit near the boundary between two ODC types. For example, a missing null check could be classified as either Checking (the validation was absent) or Algorithm/Method (the computation should have handled null inputs). Which type the LLM selects depends on whether it sees only the symptom or the fix.
3. **Multi-Fault Context:** Defects4J programs contain an average of 9.2 co-existing faults per version [2]. When multiple faults co-exist, symptom evidence from one fault can contaminate the classification of another.

The 97.1% family match demonstrates that even when the specific type shifts, the pipeline consistently identifies the correct broad category of defect. This shows robust semantic consistency in the classification approach [4], [18].

6.4 Project Coverage

The preliminary 34 bugs span 17 Defects4J projects, providing initial coverage across the full project spectrum. The distribution of bugs per project in this sample is not uniform, as we prioritized diversity across projects over depth within any single project for the initial analysis. The full experiment will cover all active bugs in every project.

6.5 Summary

The preliminary results demonstrate that the pipeline produces semantically consistent classifications. The 73.5% strict match, 94.1% top-2 match, and 97.1% family match rates fall well within the expected range for ODC classification, where perfect agreement is neither expected nor required. The type-shift patterns are scientifically explainable through evidence asymmetry, ODC boundary ambiguity, and multi-fault contamination.

Chapter 7

Conclusion

This chapter summarizes our work, discusses its limitations, and outlines future research directions.

7.1 Summary

This thesis presents an automated pipeline for classifying Defects4J bugs into Orthogonal Defect Classification (ODC) defect types using Large Language Model reasoning guided by a scientific debugging protocol. The key contributions are:

1. An end-to-end pipeline that collects multi-modal evidence from Defects4J artifacts and produces machine-readable ODC classifications with full reasoning traces.
2. A dual-mode evidence system (pre-fix and post-fix) that mirrors ODC's opener/closer model and enables built-in evaluation.
3. A structured prompting approach that combines ODC taxonomy constraints with Zeller's scientific debugging methodology.
4. A 4-tier evaluation framework that measures classification agreement at multiple granularity levels.

Preliminary results from 34 bug pairs show 73.5% strict match, 94.1% top-2 match, and 97.1% family match, demonstrating that the pipeline produces semantically consistent classifications that fall within the expected range for ODC expert agreement.

7.2 Limitations

The current work has several limitations that should be considered:

- **LLM Reproducibility:** Different LLM providers and model versions may produce different classifications for the same bug. Temperature settings and model updates can affect consistency across runs.
- **Single-Model Classification:** The current pipeline uses a single LLM for each classification. There is no consensus mechanism across multiple models or multiple runs of the same model.
- **No Ground-Truth Labels:** Since Defects4J does not include human-labeled ODC types, there is no external ground truth to validate individual classifications against. Our evaluation relies on pre-fix vs post-fix agreement as a proxy for correctness.
- **Java Only:** The pipeline is designed specifically for Defects4J, which contains only Java projects. Extending to other programming languages or bug datasets would require adaptation of the evidence collection module.
- **Partial Dataset Coverage:** The preliminary results cover only 34 bugs. The full 864+ bug analysis is still in progress.

7.3 Future Work

Several research directions could extend and improve this work:

- **Multi-Model Consensus:** Running multiple LLMs or multiple runs on the same bug and using voting or aggregation to produce more reliable classifications [5].
- **Cross-Language Extension:** Adapting the pipeline to work with other bug benchmark datasets beyond Defects4J, covering different programming languages and project types.
- **Automated Program Repair Integration:** Using ODC classifications to select and guide appropriate repair strategies, connecting the classification output to automated program repair tools [6].
- **Longitudinal Process Analysis:** Studying how ODC type distributions change over time within a project to identify quality trends and process improvements [3].

- **Human Expert Validation:** Conducting a formal study where human ODC experts label a subset of Defects4J bugs to create a ground-truth validation set for the pipeline.

7.4 Closing Remarks

Defect classification is a fundamental building block of software quality assurance. By combining the rigor of ODC's taxonomy with the reasoning capabilities of modern LLMs and the structure of scientific debugging, this thesis provides a practical, scalable approach to a problem that has traditionally required expensive manual analysis. The preliminary results suggest that LLM-driven ODC classification is a viable and promising direction for making bug triage faster, more consistent, and more informative.

References

[1] R. Andrade, C. Teixeira, N. Laranjeiro, and M. Vieira, "An Empirical Study on the Classification of Bug Reports with Machine Learning," *arXiv preprint*, 2025. arXiv: 2503.00660.

[2] D. Callaghan, *Mining Bug Repositories for Multi-Fault Programs*, GitHub, defects4j-mf, 2024.

[3] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, "Not All Bugs Are the Same: Understanding, Characterizing, and Classifying Bug Types," *Journal of Systems and Software*, vol. 153, pp. 1–18, 2019. DOI: 10.1016/j.jss.2019.03.002

[4] R. Chillarege et al., "Orthogonal Defect Classification – A Concept for In-Process Measurements," *IEEE Transactions on Software Engineering*, vol. 18, no. 11, pp. 943–956, 1992. DOI: 10.1109/32.177364

[5] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Large Language Models for Issue Report Classification," in *Ital-IA 2024, CEUR Workshop Proceedings*, 2024.

[6] T. Durieux, M. Martinez, M. Monperrus, and J. Wuttke, "Automatic Repair of Real Bugs in Java: A Large-Scale Experiment on the Defects4J Dataset," *arXiv preprint*, 2018. arXiv: 1811.02429.

[7] X. Gao et al., "RCEGen: A Generative Approach for Automated Root Cause Analysis," *MDPI Computers*, vol. 4, no. 4, p. 29, 2024.

[8] T. Hirsch and B. Hofer, "Using Textual Bug Reports to Predict the Fault Category of Software Bugs," *Array*, vol. 15, p. 100 189, 2022. DOI: 10.1016/j.array.2022.100189

[9] B. Huang and V. Ng, "AutoODC: Automated Generation of Orthogonal Defect Classifications," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2011. DOI: 10.1109/ASE.2011.6100095

[10] IBM, "Orthogonal Defect Classification v 5.2 for Software Design and Code," IBM Research, Tech. Rep., 2013.

[11] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055

[12] S. Kang, B. Chen, S. Yoo, and J.-G. Lou, "Explainable Automated Debugging via Large Language Model-driven Scientific Debugging," *arXiv preprint*, 2023. arXiv: 2304.02195.

[13] A. Koyuncu, "Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study," *ACM Transactions on Software Engineering and Methodology*, 2025. DOI: 10.1145/3712000

[14] Y. Nong et al., "Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities," *arXiv preprint*, 2024. arXiv: 2402.17230.

[15] X. Pan et al., "Understanding Defects in Generated Codes by Language Models," in *CASCON 2024*, arXiv:2408.13372, 2024.

[16] M. N. Rafi, A. R. Chen, T.-H. P. Chen, and S. Wang, "Revisiting Defects4J for Fault Localization in Diverse Development Scenarios," *arXiv preprint*, 2024. arXiv: 2402.13040.

[17] V. Sobreira, T. Durieux, F. Madeiral, M. Monperrus, and M. d. A. Maia, "Dissection of a Bug Dataset: Anatomy of 395 Patches from Defects4J," in *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2018, pp. 130–140. DOI: 10.1109/SANER.2018.8330203

[18] F. Thung, D. Lo, and L. Jiang, "Automatic Defect Categorization," in *Proceedings of the Working Conference on Reverse Engineering (WCRE)*, IEEE, 2012. DOI: 10.1109/WCRE.2012.45

[19] A. Van der Spuy and B. Fischer, "An Anatomy of 488 Faults from Defects4J Based on the Control- and Data-Flow Graph Representations of Programs," *arXiv preprint*, 2025. arXiv: 2502.02299.

[20] J. Xuan, M. Monperrus, et al., "Better Test Cases for Better Automated Program Repair: A Manual Inspection of Defects4J Bugs and Its Implications," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, ACM, 2017, pp. 831–841. DOI: 10.1145/3106237.3106295

[21] A. Zeller, *Why Programs Fail: A Guide to Systematic Debugging*, 2nd. Morgan Kaufmann, 2009.